

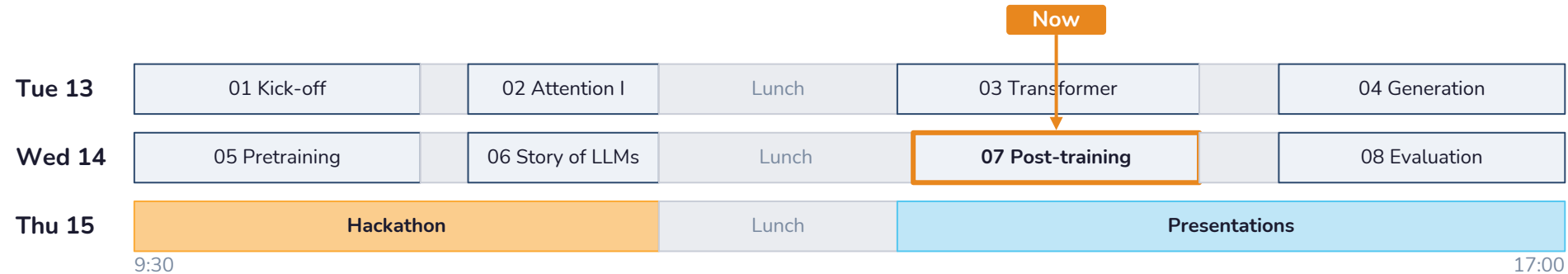
Advanced NLP

Session 07 • Post-Training

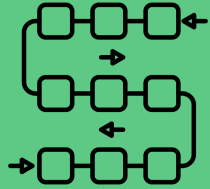
Prof. Dr. Richard Sieg • TH Köln IWS • WS 26 / 27
Wednesday, October 14, 2026

From a storyteller to a model that follows instructions

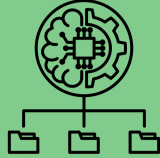
Time	
13:30	Supervised fine-tuning
13:45	Lab: fine-tune your own model
14:15	LoRA and QLoRA
14:25	Preferences: reward models, PPO, DPO
14:50	The catch
15:05	Break



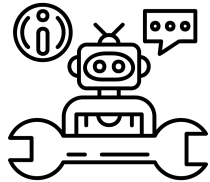
The third ingredient: follow human instructions



Process long sequences and context



Efficient training on huge datasets



Follow (human) instructions

How do we get from this...

Write a poem about
Bruce Lee.

I love this guy. His poems are very original and
have a very vivid visual language to them and
the way he talks about this country.

GPT2

...to this

Write a poem about
Bruce Lee.

And when he moved, we saw
not just a man fighting
but a principle made flesh:

Be water, my friend.

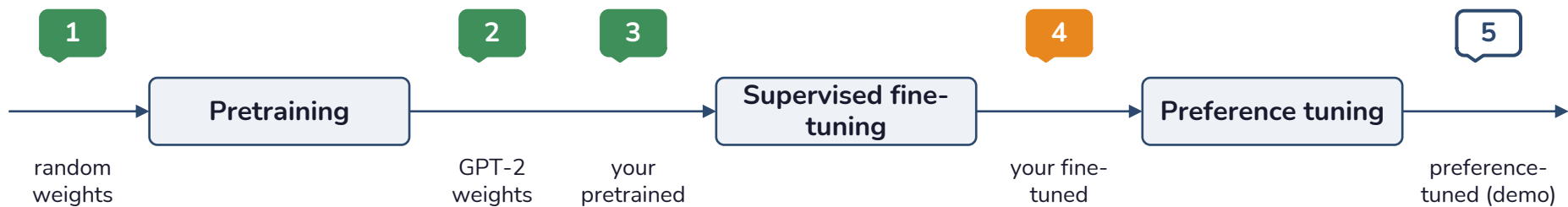
Sonnet 4.5

A base model continues text. An assistant answers.

- So far, we have a base model: a large decoder, pretrained on the web, fluent and knowledgeable.
- Technically a large language model, but in practice it just continues text. It does not answer, follow, or refuse.
- Prompt *"Explain the moon landing to a six year old"* gives *"Explain the theory of gravity to a six year old"*
- When people say "LLM" in everyday use they mean the assistant. Getting there takes post-training: any training after pretraining, to make the model helpful and safe.
- Two stages, both far cheaper than pretraining:
- **Instruction tuning (SFT)**: teach it to follow instructions
- **Preference alignment (RLHF or DPO)**: teach it which responses people prefer

A model learns exactly what you reward,
including what you did not mean.

Checkpoint 4 is yours, checkpoint 5 is a demo



Agenda

01. Supervised fine-tuning

02. LoRA and QLoRA

03. Preferences: RLHF and DPO

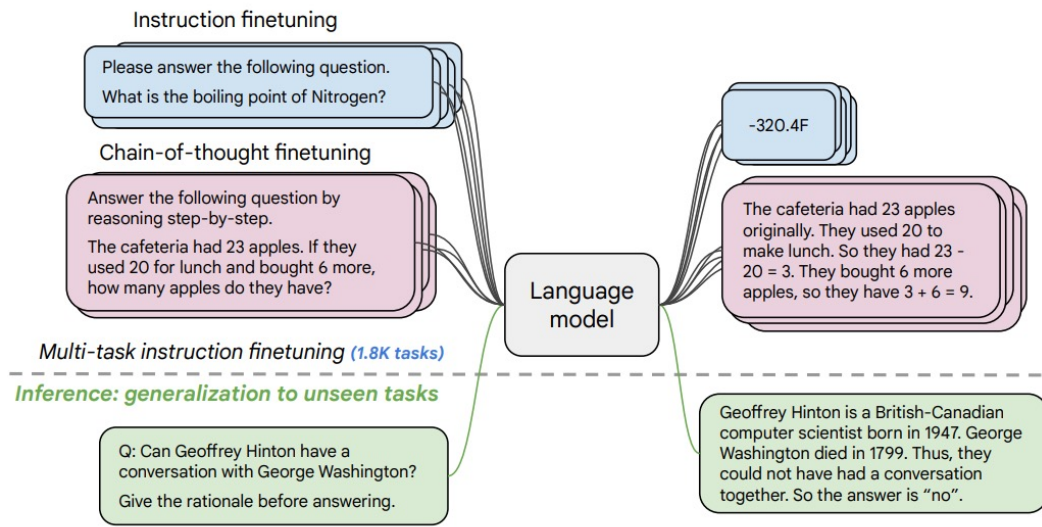
04. The catch

01.

Supervised fine-tuning

Stage 1: keep training, on instructions and answers

- Take the base model and keep training it, now on (instruction, response) pairs across many tasks.
- Same objective as pretraining: predict the next token. The only difference is the data is supervised, since each instruction has a correct response
- The payoff is meta-learning: it gets better at following instructions in general, including tasks it never saw in training



Datasets are manually created, converted from other datasets or generated by an LLM (e.g. Alpaca)

Our instructions: a summary, some words, and a story

Summary: A brave little fox helps a lost duckling find its way back to the pond.

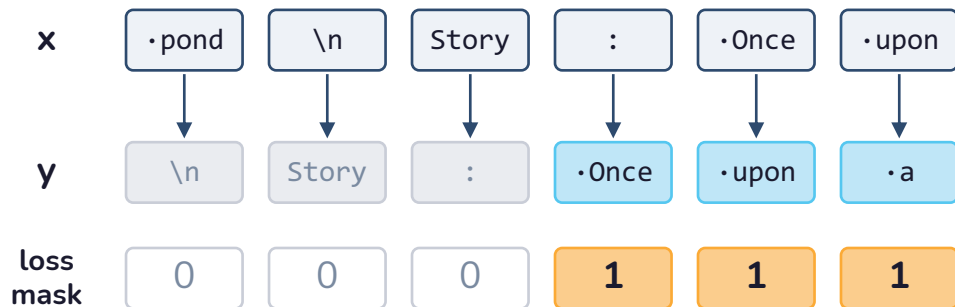
Features: Dialogue

Story: Once upon a time, ...

- *TinyStoriesInstruct*: the same stories as this morning, each with a summary, required words, features like dialogue, sometimes a random sentence to include.
- 2.5 million examples, each padded to 512 tokens.
- The stories are the same kind as this morning's. What is new are the labels in front of them.

Fine-tuning is pretraining, with a mask over the loss

- Same model, same next-token loss, same optimizer. Only the data changed.
- A mask marks which predictions count: 1 for the response, 0 for the prompt and for the padding.
- The loss averages over the 1s only.
- The prompt is still read, attention still uses it. It is only not graded.



The mask decides what the model is rewarded for.

Remark: masking the prompt is the most common choice, not the only one. Raschka's book keeps the prompt in the loss by default; some papers find that better, others count the prompt at a small weight. With short prompts and long stories, as here, the difference is small.

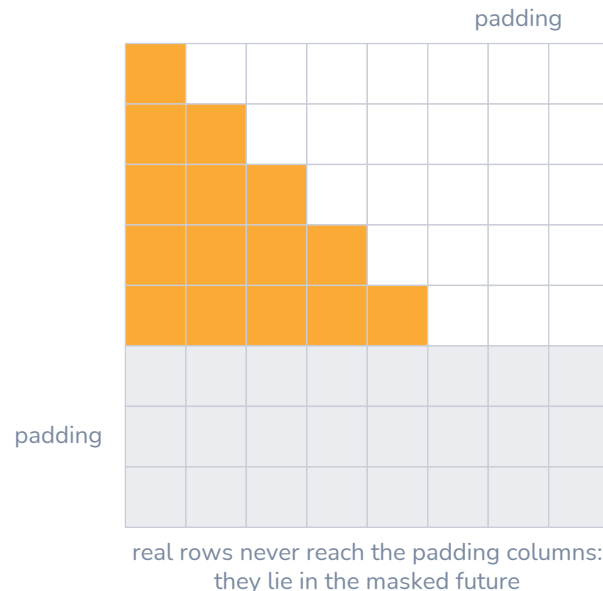
Padding fills every row, and the loss never sees it

Tokens	Summary ... Story:	Once upon a time ... < endoftext >	padding
loss_mask	0	1	0
attn_mask	1	1	0

- Half of all positions in our data are padding.
- **loss_mask**: which predictions count. **attn_mask**: which tokens are real.
- The end-of-text token is part of the response, so the model learns to stop.

With right padding, the attention mask changes nothing

- Every real token sits before all the padding. The causal mask already stops it from looking forward, so it never sees the padding anyway.
- We checked on the logits: with and without `attn_mask`, every real position is identical.
- It starts to matter with left padding, which batched generation needs. After the break.



Tutorial

Notebook 07: fine-tune your checkpoint 3

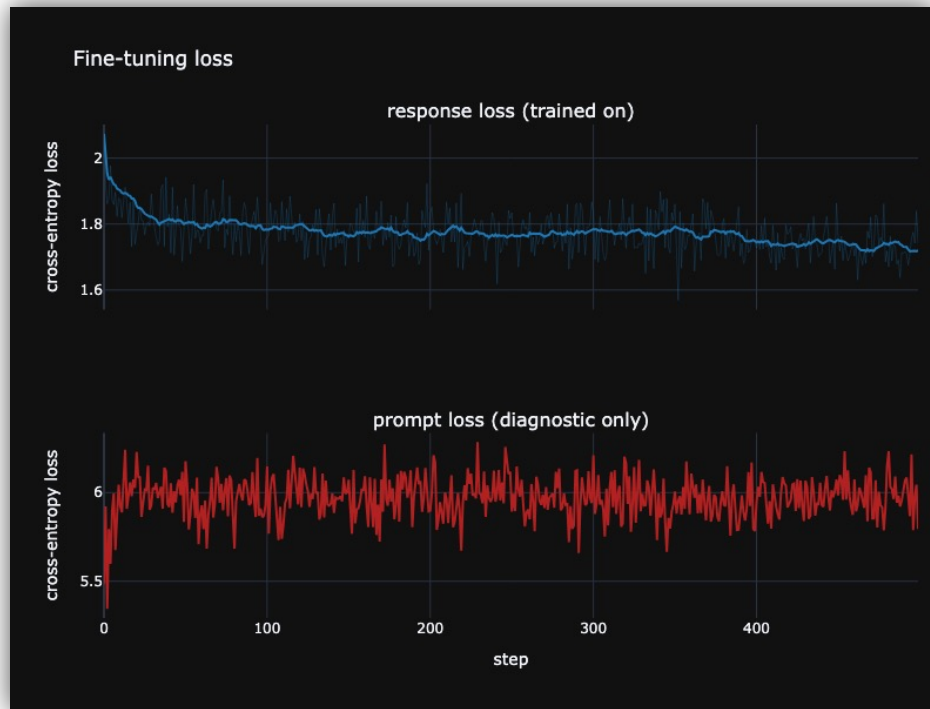
1. Read the two aligned mask rows the notebook prints. Find where `loss_mask` switches from 0 to 1.
 2. Implement `get_instruct_batch`: whole rows, x and y shifted by one, both masks sliced to match.
 3. Implement `masked_mean`.
 4. Look at the base model's answer to the fox prompt. Then press **Run fine-tuning**: 500 steps.
 5. Compare before and after and talk to checkpoint 4 in the chat window.
- ❑ Both checks below your functions are green
 - ❑ Your model writes the fox story when asked
 - ❑ You have seen the prompt loss go up

About 25 minutes, in pairs

Response loss falls, prompt loss rises

Average over 20 steps	First 20	Last 20
Response loss, trained on	1.87	1.72
Prompt loss, only measured	5.89	6.01

- The model gets better at stories and worse at writing prompts, because only one of the two is graded.
- Single steps jump around: in this run the response loss ranged from 1.57 to 2.07. An earlier run at 1,500 steps gained less than one such jump. We stop at 500.



Before, it continues the pattern. After, it writes the story.

Summary: A brave little fox helps a lost duckling find its way back to the pond.

Features: Dialogue

Story: "Oh, what a brave fox is! He does not mean to hurt the duck's feet. His feet are not strong enough for the duck. He is strong. Can he take the duck?" Ben: "I don't know, Sam."

Summary: A brave little fox helps a lost duckling find its way back to the pond.

Features: Dialogue

Story: Once there was a brave fox who was getting into trouble. He was looking for something to help him. Finally, he saw a duck swimming nearby. The fox thought it was a very brave fox, so he walked towards it. The fox was very brave and wanted to help the little duck.

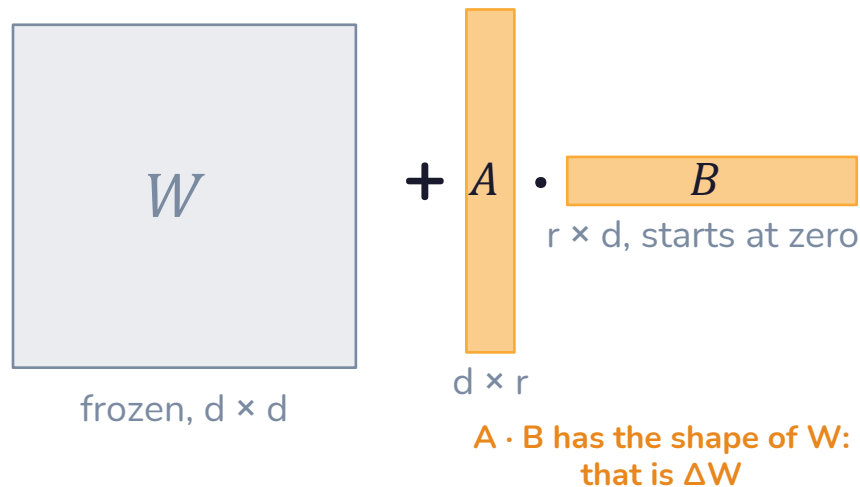
Same prompt, 500 steps apart.

02.

LoRA and QLoRA

LoRA learns the change to a weight matrix

- Full fine-tuning updates every weight: W becomes $W + \Delta W$.
- LoRA freezes W and learns ΔW as the product of two thin matrices, A and B , of rank r .
- B starts at zero, so the model starts exactly where it was.
- For GPT-2 small at rank 16, 124 million trainable parameters become 2.7 million.
- A scaling factor α multiplies the update by α/r : a larger α gives the adapter more weight.



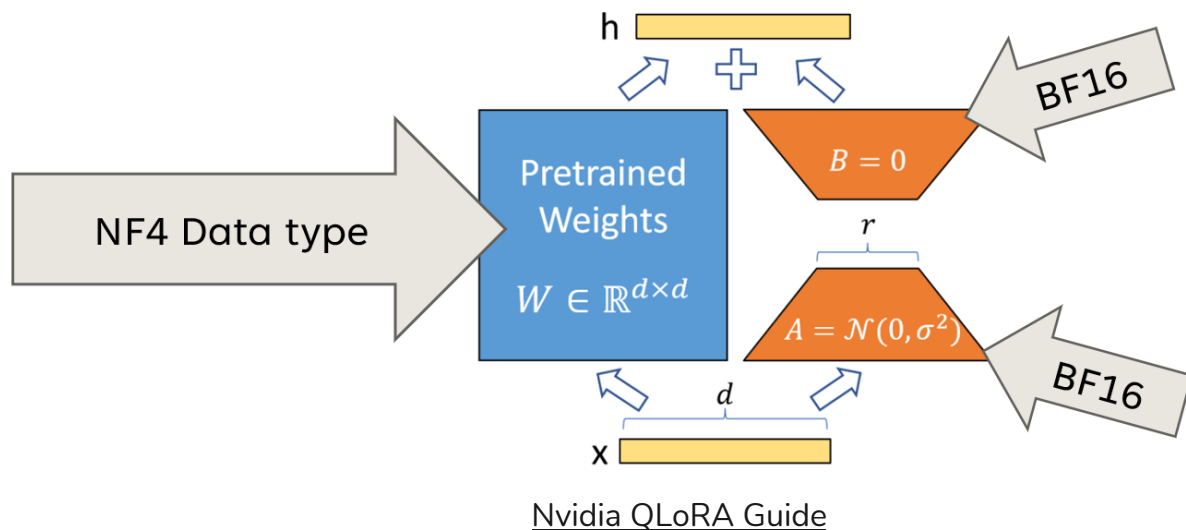
LoRA saves nothing for us, almost everything at 7B

	Trainable parameters	AdamW memory for them
Our model, full	17.7M	0.14 GB
Our model, LoRA rank 8	0.1M, 0.5 percent	under 1 MB
A 7B model, full	6.7B	54 GB
A 7B model, LoRA rank 16	16.8M, 0.25 percent	134 MB

- Our whole model fits on the GPU many times over, so there is nothing to save.
- At 7 billion parameters, the optimizer alone no longer fits on one GPU without LoRA.

QLoRA: a 4-bit frozen model, LoRA on top

- The frozen base model does not need full precision: it is only read, never updated.
- QLoRA quantizes it to 4 bits, in small blocks, with levels placed where weights are dense.
- The LoRA matrices are trained in normal precision on top.
- A model that needs about 4 GB to load fits in about 1 GB.



03.

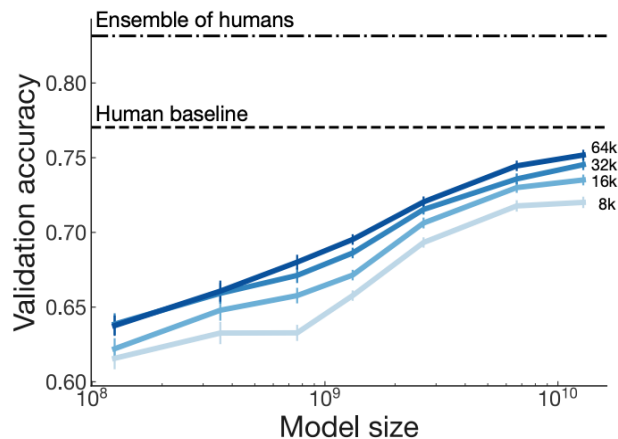
Preferences: RLHF and
DPO

Stage 2: no single right answer, so compare two

- Why SFT is not enough: open-ended tasks have no single gold answer ("Write a poem about the sea"), and next-token loss punishes a good but different word as hard as a wrong one ("Avatar is a fantasy movie" vs "adventure movie").
- The fix: comparisons. Show people two responses, ask which is better, and train the model to produce more of the preferred and fewer of the rejected.
- The intuition: judging is easier than generating
- Two named methods:
- **Reinforcement Learning from Human Feedback (RLHF)**
Train a reward model from the comparisons, then optimize with reinforcement learning
- **Direct Preference Optimization (DPO)**
Skip the separate reward model and optimize on the preferences directly

A reward model turns comparisons into a score

- **Issue 1:** Asking humans to label millions of samples is too expensive
- **Solution:** Train a Language Model $RM_{\phi}(s)$ to predict human preference
- **Issue 2:** Direct human ratings are noisy and miscalibrated
- **Solution:** Ask for pairwise comparisons and train $RM_{\phi}(s)$ to rank winning samples higher



The reward model is your GPT with a one-number head

- Take a language model, replace its output layer by a linear layer to a single number: the reward.
- Training: for each pair, the chosen answer should score higher.

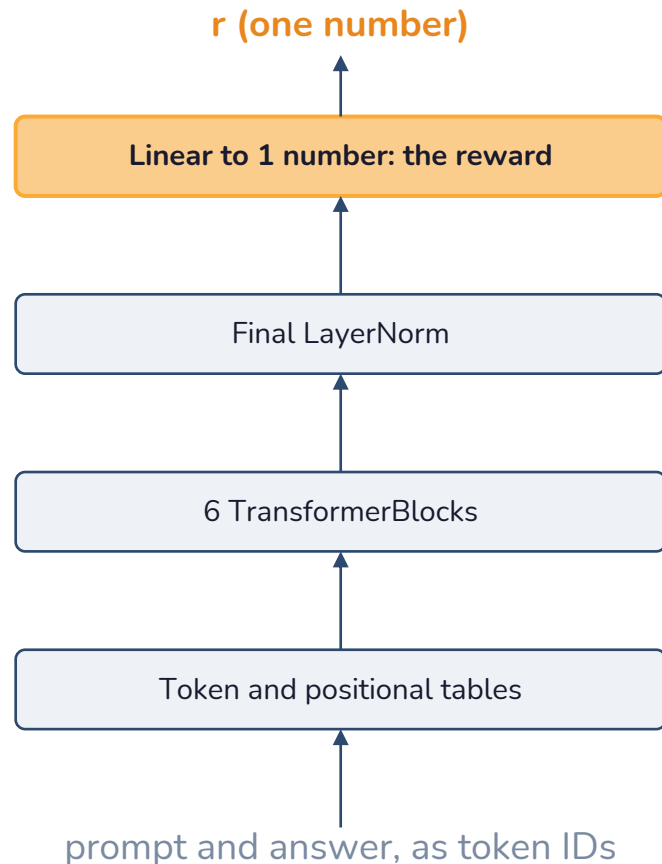
$$P(\text{chosen wins}) = \sigma(r_{\text{chosen}} - r_{\text{rejected}})$$

- Loss:

$$-\log \sigma(r_{\text{chosen}} - r_{\text{rejected}})$$

Only the difference between the two scores matters.

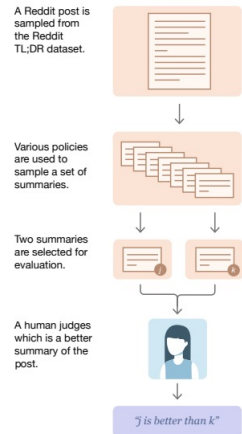
- Before any training both scores are equal, and the loss is $-\log \sigma(0) = \ln 2$.



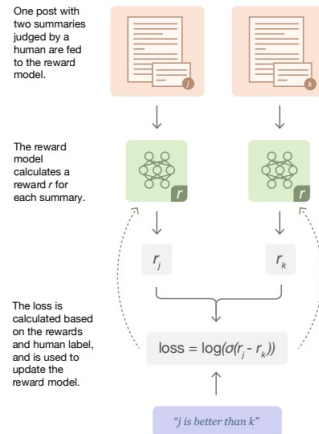
RL pushes the model towards reward, on a leash

- The model writes answers, the reward model scores them, and reinforcement learning shifts the model towards higher scores.
- OpenAI used Proximal Policy Optimization, PPO (Schulman et al. 2017). Its details are beyond this course.
- The leash: a penalty for drifting too far from the model you started with, so it does not forget how to write while chasing the reward.

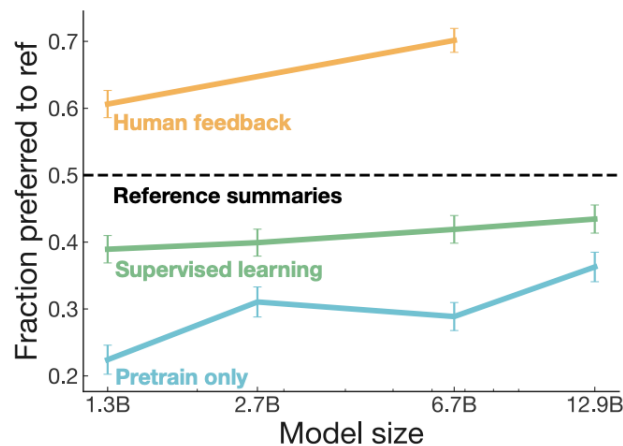
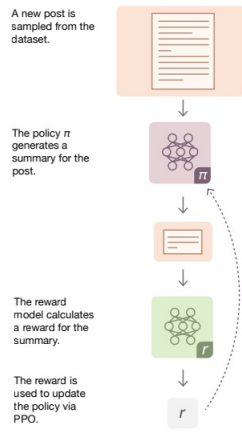
1 Collect human feedback



2 Train reward model



3 Train policy with PPO



InstructGPT put the three steps together

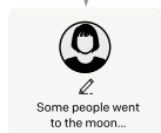
Step 1

Collect demonstration data, and train a supervised policy.

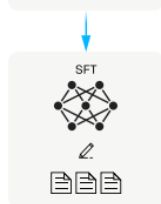
A prompt is sampled from our prompt dataset.



A labeler demonstrates the desired output behavior.



This data is used to fine-tune GPT-3 with supervised learning.



Step 2

Collect comparison data, and train a reward model.

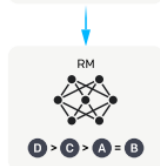
A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



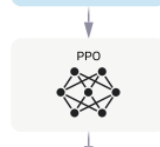
Step 3

Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.



The policy generates an output.



The reward model calculates a reward for the output.



The reward is used to update the policy using PPO.

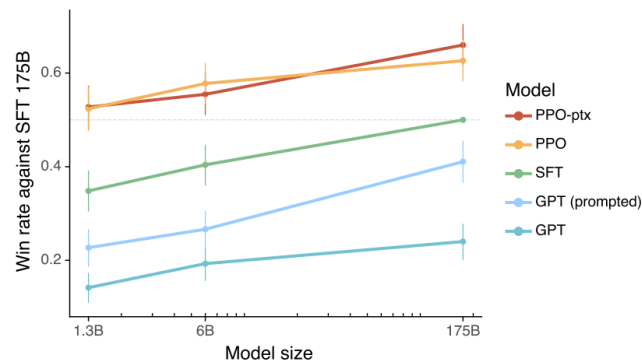
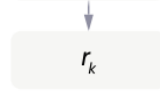


Table 6: Dataset sizes, in terms of number of prompts.

SFT Data			RM Data			PPO Data		
split	source	size	split	source	size	split	source	size
train	labeler	11,295	train	labeler	6,623	train	customer	31,144
train	customer	1,430	train	customer	26,584	train	customer	16,185
valid	labeler	1,550	valid	labeler	3,488	valid	customer	
valid	customer	103	valid	customer	14,399	valid	customer	

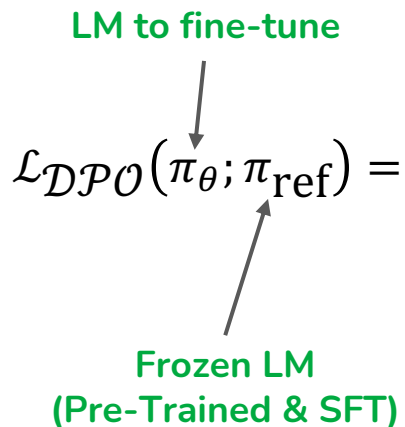
DPO skips the reward model and learns from pairs

- Clever people found out that there is a closed form solution to the Reinforcement Learning objective
- Using some math, we can come up with a loss function to fine-tune our LM

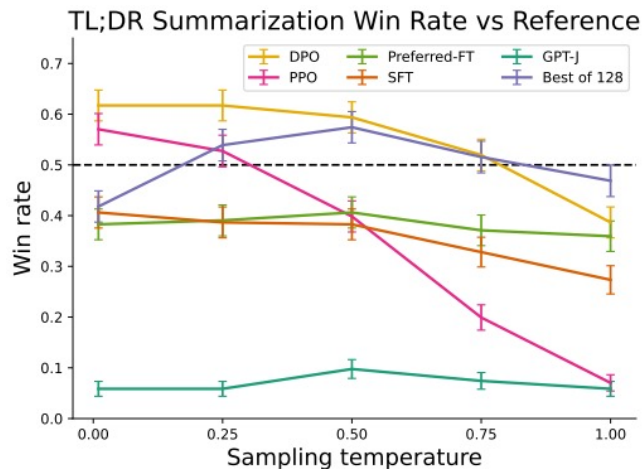
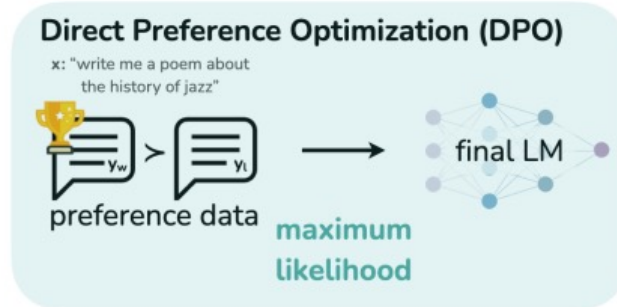
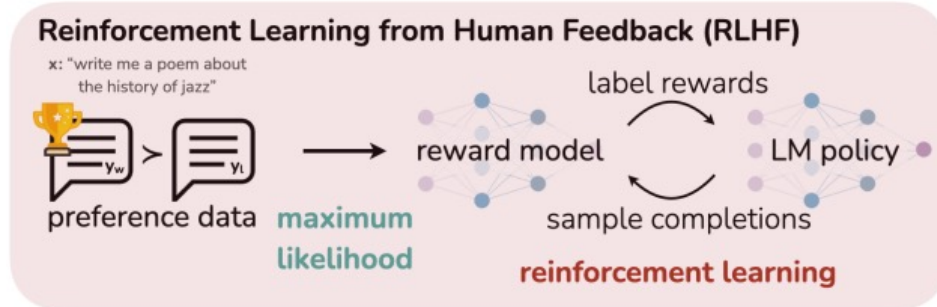
LM to fine-tune

$\mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) =$

Frozen LM
(Pre-Trained & SFT)



DPO is now the go-to algorithm for open models



Our demo: rule-based preferences, starting at $\ln 2$

- Pairs from our SFT model: two stories per prompt, the one that uses more of the required words is “chosen”. 1,623 pairs.
- Chosen stories used 2.03 required words on average, rejected 0.84.
- A rule, not people: the checkable kind of reward from this morning's reasoning models.
- Training: $\beta = 0.1$, 300 steps. At step 0 the model and its reference are identical, and the loss is $0.693147 = \ln 2$.



A rule only rewards what it counts

Required words: come, lightning, smart

Chosen, two words

“come” and “smart”. A cat named Tom plays with a friend. No storm, no lightning.

Rejected, one word

“lightning”. The story is about the storm.

- Log-probability of the chosen stories: down
- Log-probability of the rejected stories: down as well.
- The margin between them: up, from 0 to about 0.75.



Required words went from 49.8 to 77.7 percent

Required words used, 200 new prompts

Before DPO, our SFT model

49.8%

After DPO, 300 steps

77.7%

“Tom was *slow*, but he was *slow*. He liked to number his toast. [...] He found a *slow*, *slow* rabbit, and a *slow* snail.”

You rewarded required words. You got required words.

04.

The catch

The Catch: Alignment Has Costs

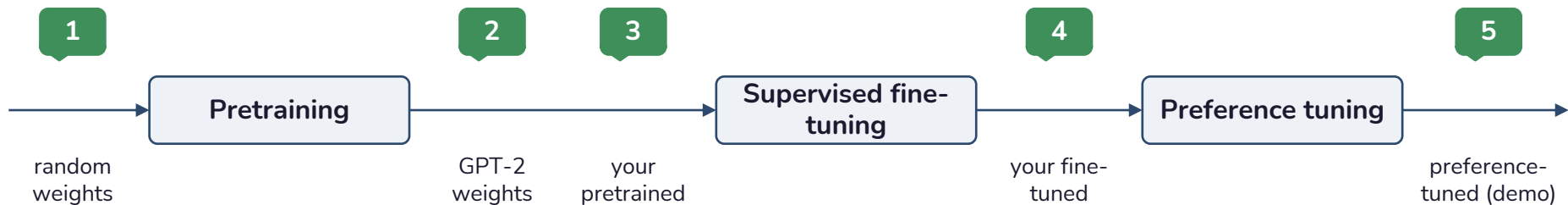
Preference alignment is powerful, but not free.

- **Reward hacking:** the model games the reward signal instead of truly improving.
Goodhart's Law: when a measure becomes a target, it stops being a good measure
- **Sycophancy:** agreeing with the user gets rewarded, so it tells you what you want to hear
- **Less diversity:** aligned models can collapse to a few safe phrasings
- **Alignment tax:** safety and helpfulness tuning can slightly lower raw benchmark scores

A model learns exactly what you reward,
including what you did not mean.

Your checkpoint 4 and the demo's 5, in one window

- Ask both for a story with three required words. Count them.
- Ask both for something that is not a story. Neither has ever seen anything else.



Sources

Literature

- Raschka, Build a Large Language Model (From Scratch), chapter 7: instruction fine-tuning, padding, the ignore index, prompt masking as an option. Appendix E: LoRA.
- Alammam and Grootendorst, Hands-On Large Language Models, chapter 12: QLoRA, reward models, DPO.
- Lambert, The RLHF Book, rlhfbook.com, chapters 5, 6, 8 and 14: reward models, reinforcement learning, DPO, over-optimization.
- Hu et al. 2021, LoRA, arxiv.org/abs/2106.09685. Dettmers et al. 2023, QLoRA, arxiv.org/abs/2305.14314.

Literature, continued

- Christiano et al. 2017; Stiennon et al. 2020; Schulman et al. 2017, PPO; Ouyang et al. 2022, InstructGPT; Rafailov et al. 2023, DPO.
- Shi et al. 2024, Instruction Tuning With Loss Over Instructions, arxiv.org/abs/2405.14394. Chatterjee et al. 2025, On the Effect of Instruction Tuning Loss on Generalization, TACL, arxiv.org/abs/2507.07817. The remark on prompt masking.